ZHAW School of Engineering
Institute of Mechatronic Systems

Betaflight – Offline Controller Tuning

Data-Driven Tuning of Rate and Altitude Controllers for FPV Drones

Bachelor Thesis

Yuri Bianchi, Dario Jurietti and Janick Dort

Supervisor: Michael Peter (pmic)
Co-Supervisor: Prof. Dr. Ruprecht Altenburger (altb)

March 13, 2026

Declaration of Originality

We hereby declare that this thesis is our own work and has been composed solely by the undersigned. We have not used any sources or aids other than those indicated. All passages taken verbatim or in substance from published or unpublished works of others have been clearly marked as such. This thesis has not been submitted, in whole or in part, for any other degree or examination at this or any other institution.

Winterthur, March 13, 2026

Yuri Bianchi

Dario Jurietti

Janick Dort

Zusammenfassung

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Abstract

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Contents

Preface

This thesis was carried out during the spring semester 2026 at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. It continues the work started in the preceding project thesis conducted in the fall semester 2025.

We would like to thank our supervisor, Michael Peter, for...

0.1. Introduction

First-Person-View (FPV) racing drones require precise and responsive control to achieve stable flight behaviour, suppress disturbances such as propeller wash, and maintain agility during rapid manoeuvres. The flight performance strongly depends on the tuning of the flight controller's feedback loops. In particular, the parameters of the angular rate controller determine how quickly and accurately the drone reacts to pilot inputs and external disturbances.

In practice, controller tuning is still often performed manually through iterative trial-and-error procedures. This requires extensive flight testing and considerable piloting experience, making the process time-consuming and difficult to reproduce. Although modern flight controller firmware such as Betaflight provides extensive configuration options and detailed flight data logging, systematic tuning remains a challenge.

One commonly used tool for offline tuning is the PID-Toolbox. In this approach, flight logs are recorded during normal flights and later analysed to derive suitable controller parameters. However, the system excitation depends entirely on the pilot's inputs and the natural flight behaviour of the drone. As a result, not all relevant frequency ranges are sufficiently excited, which can limit the quality of the measurements.

In the approach proposed in this thesis, a chirp excitation signal is injected directly into the control loop. This allows the system to be excited over a wide range of frequencies in a controlled and systematic way, resulting in higher-quality measurements for system identification and controller tuning.

A previous project introduced a proof-of-concept approach for offline tuning of angular rate controllers based on recorded flight data. In the preceding project thesis, this concept was restructured and modularised and now serves as the foundation for the present work.

The objective of this bachelor thesis is to further refine and extend this approach into an offline tuning framework for multiple control loops used in Betaflight-based flight controllers. In addition to the existing angular rate controller tuner, the framework will be extended with code modules for tuning the angle controller, the altitude hold controller, and the position controller. This extends the tuning framework beyond the inner rate control loop and enables the analysis and tuning of higher-level control layers.

The framework will be implemented primarily in MATLAB and Python and will provide tools for analysing flight logs, estimating system dynamics, and generating controller parameters without requiring repeated flight tests. The software will be developed as an open-source library with accompanying documentation for the FPV community.

By providing a structured, data-driven approach to controller tuning, this project aims to simplify the tuning process and improve flight performance.

0.2. Theory

In this chapter, the theoretical background of the approach used for tuning the different control loops of the flight controller is introduced. Since the fundamental working principle is identical for all loops, the general concepts are explained here before addressing the individual loops in the methodology section.

To support the theoretical concepts, a simplified simulation model is introduced. The simulation is gradually developed throughout this chapter, allowing the reader to follow the modelling process step by step and to better understand the behaviour of the control system. This approach illustrates the influence of the controller parameters and provides an intuitive understanding of the tuning process.

The detailed application of these principles to the different control loops implemented in Betaflight is discussed in the following chapter.

0.2.1. Chirp Signal

A chirp signal is a sinusoidal signal whose frequency varies continuously over time. It is widely used in system identification to excite a broad range of frequencies in a single measurement, making it possible to estimate the frequency response $G(\omega)$ efficiently.

0.2.1.1. Mathematical Definition

The instantaneous value of the chirp is defined as

$$x(t) = \sin(\arg(t)),$$

where the phase argument

$$\arg(t) = 2\pi \int_0^t f(\tau)\, d\tau$$

represents the **instantaneous phase** of the signal. The derivative of this phase yields the **instantaneous frequency**

$$f(t) = \frac{1}{2\pi} \frac{d(\arg(t))}{dt}.$$

In practical implementations such as **Betaflight**, the phase $\arg(t)$ is wrapped using the modulo operation

$$\arg(t) \bmod 2\pi,$$

so that it resets to 0 whenever a full 2π rotation is reached. This keeps the numerical values bounded and results in the sawtooth-shaped `sinarg` signal often observed in practice.

0.2.1.2. Common Frequency Profiles

Linear chirp:

The frequency increases linearly over time:

$$k = \frac{f_1 - f_0}{T}$$

$$f(t) = f_0 + kt$$

$$\arg(t) = 2\pi \left(f_0 t + \tfrac{1}{2} k t^2 \right)$$

Exponential chirp (as used in Betaflight):

The frequency grows exponentially from the starting frequency f_0 to the ending frequency f_1 over the time interval T:

$$f(t) = f_0 \left(\frac{f_1}{f_0} \right)^{t/T}$$

$$\arg(t) = \frac{2\pi T f_0}{\ln(f_1/f_0)} \left[\left(\frac{f_1}{f_0} \right)^{t/T} - 1 \right]$$

These functions define the **phase trajectory** $\arg(t)$ used in simulations and in the `apply_rotfiltfilt` method for time-dependent frequency shifting of chirp signals.

0.2.1.3. Lag Filter for Chirp Input Shaping

Prior to injection into the control loop, the chirp signal is conditioned using a filter to compensate for the differentiating behaviour of the controller effort at low frequencies. This differentiating characteristic leads to an increasing gain toward lower frequencies, which would otherwise result in excessive motor excitation at the beginning of the chirp sweep. Such behaviour may cause actuator saturation and thereby degrade the quality of the system identification data.

To mitigate this effect, the filter attenuates low-to-mid frequency components of the chirp signal before injection. As a result, the combined response of the filter and controller produces a more uniform excitation level across the frequency range of interest.

The filter is configured with a pole at **3 Hz** and a zero at **30 Hz**, yielding the following transfer function:

$$H(s) = \frac{1 + s/\omega_z}{1 + s/\omega_p}$$

where

$$\omega_z = 2\pi \cdot 30 \text{ rad/s}$$

represents the zero frequency and

$$\omega_p = 2\pi \cdot 3 \text{ rad/s}$$

represents the pole frequency.

The pole introduces the lag characteristic by reducing the magnitude for frequencies above its cutoff frequency. Conversely, the zero limits this attenuation by introducing magnitude amplification above its own cutoff frequency. However, since the pole frequency is lower than the zero frequency, the pole's contribution dominates over the primary control bandwidth, resulting in an overall lag filter behaviour.

It is important to note that in the Betaflight implementation, the pole frequency ω_p must not be set higher than the zero frequency ω_z. Otherwise, the filter would behave as a lead filter, amplifying high-frequency components and potentially causing excessive motor excitation or hardware damage.

0.2.2. Signal Processing

As already mentioned in the section *Chirp Signal for System Identification*, the transfer function of the whole system from the target angle to the measured angle can be estimated directly from the input and output signals. However, due to disturbances and measurement noise, the estimated transfer function would not accurately reflect the true system dynamics. Therefore, additional processing steps are required to isolate the transfer function dynamics from these effects.

To illustrate the influence of the applied signal processing steps, a simulation is used as a reference. The simulation is based on a predefined transfer function. For this example, a PT1 system is used. The transfer function is given by

$$G(s) = \frac{1}{0.1s + 1}.$$

If the system is analysed in the frequency domain using a Bode plot, the magnitude decreases with increasing frequency and the phase shifts from $0°$ at low frequencies to $-90°$ at high frequencies.

To measure the transfer function, a chirp signal is applied to the input of the system and the corresponding output signal is recorded. Disturbances and noise are added to the output signal in order to simulate realistic measurement conditions.

The transfer function is then estimated using the spectral relation

$$G(\omega) = \frac{S_{yu}(\omega)}{S_{uu}(\omega)}.$$

0.2.2.1. Estimate Frequency Response

For a meaningful estimation of the transfer function, both the frequency response and the coherence between the input and output signals are calculated. The coherence provides a measure of how strongly the output signal is linearly related to the input signal at each frequency. Values close to one indicate a strong linear relationship and therefore a reliable transfer function estimate, whereas low coherence values indicate that noise or nonlinear effects dominate the measurement.

In the frequency domain, the relation between the input and output of a linear time-invariant (LTI) system can be expressed as

$$G(\omega) = \frac{Y(\omega)}{U(\omega)}.$$

Here, $U(\omega)$ and $Y(\omega)$ denote the Fourier transforms of the input signal $u(t)$ and the output signal $y(t)$, respectively. This ratio describes how each frequency component of the input is scaled and phase-shifted by the system.

In practice, however, the measured output signal is affected by disturbances. The measured output can therefore be written as

$$Y(\omega) = G(\omega)U(\omega) + D_y(\omega),$$

where $D_y(\omega)$ represents an additive disturbance at the output. If the transfer function were estimated directly by dividing $Y(\omega)$ by $U(\omega)$, the disturbance term would directly influence the result

$$\frac{Y(\omega)}{U(\omega)} = G(\omega) + \frac{D_y(\omega)}{U(\omega)}.$$

This expression shows that a direct division is highly sensitive to disturbances and measurement noise.

To obtain a more reliable estimate, spectral quantities based on statistical averaging are used. The cross-power spectral density between input and output and the power spectral density of the input are defined as

$$S_{yu}(\omega) = \mathbb{E}[Y(\omega)U^*(\omega)], \qquad S_{uu}(\omega) = \mathbb{E}[|U(\omega)|^2].$$

Substituting the disturbed output signal into the cross-power spectral density yields

$$S_{yu}(\omega) = \mathbb{E}[(G(\omega)U(\omega) + D_y(\omega))U^*(\omega)] = G(\omega)S_{uu}(\omega) + \mathbb{E}[D_y(\omega)U^*(\omega)].$$

If the disturbance $D_y(\omega)$ is uncorrelated with the input signal, the second term becomes zero and the expression simplifies to

$$S_{yu}(\omega) = G(\omega)S_{uu}(\omega).$$

This leads to the practical and statistically robust estimator of the transfer function

$$G(\omega) = \frac{S_{yu}(\omega)}{S_{uu}(\omega)}.$$

0.2.2.2. Welch's Method for Spectral Estimation

To estimate the spectral densities $S_{yu}(\omega)$ and $S_{uu}(\omega)$ from finite data, Welch's method is employed. The spectral densities cannot be computed directly from the expectation operator, since only finite time-domain measurements of the input and output signals are available. Instead, these quantities must be estimated from the measured data. In this work, Welch's method is used to obtain reliable estimates of the power spectral densities and cross-power spectral densities.

Let $u[n]$ and $y[n]$ denote the discrete-time input and output signals of length N. In Welch's method, the signals are divided into K overlapping segments of length L. Each segment is multiplied by a window function $w[n]$ in order to reduce spectral leakage.

The k-th windowed segment of the input signal can therefore be written as

$$u_k[n] = u[n + kD]\, w[n], \qquad n = 0, \ldots, L-1$$

where D denotes the shift between successive segments and determines the amount of overlap between segments.

For each segment, the discrete Fourier transform (DFT) is computed

$$U_k(\omega) = \sum_{n=0}^{L-1} u_k[n] e^{-j\omega n}$$

and analogously for the output signal

$$Y_k(\omega) = \sum_{n=0}^{L-1} y_k[n] e^{-j\omega n}.$$

From these Fourier transforms, the periodogram estimate of the power spectral density of the input signal for each segment is obtained as

$$P_{uu,k}(\omega) = \frac{1}{L}|U_k(\omega)|^2.$$

Similarly, the cross-periodogram between output and input is defined as

$$P_{yu,k}(\omega) = \frac{1}{L} Y_k(\omega) U_k^*(\omega).$$

Welch's method reduces the variance of the spectral estimates by averaging the periodograms over all segments. The resulting estimates of the power spectral density and the cross-power spectral density are therefore given by

$$S_{uu}(\omega) = \frac{1}{K} \sum_{k=1}^{K} P_{uu,k}(\omega)$$

and

$$S_{yu}(\omega) = \frac{1}{K} \sum_{k=1}^{K} P_{yu,k}(\omega).$$

These averaged spectral estimates are then used to compute the transfer function estimate

$$G(\omega) = \frac{S_{yu}(\omega)}{S_{uu}(\omega)}.$$

0.2.2.3. Baseband Filtering via Frequency Rotation

Measured signals are often contaminated by measurement noise and unwanted high-frequency components. These disturbances can obscure the relevant information contained in the signal and may negatively affect subsequent analysis steps. To suppress these disturbances while preserving the phase characteristics of the signal, a frequency-shifting filtering approach can be employed.

As an illustrative example, consider a sinusoidal signal corrupted by additive noise. The objective of the filtering process is to suppress the noise components while preserving the amplitude and phase of the underlying signal.

0.2.2.4. Frequency Shifting to Baseband

The filtering method is based on shifting the frequency content of the signal to baseband, applying a low-pass filter, and subsequently shifting the signal back to its original frequency range. This approach allows the filtering operation to focus on a narrow frequency region around the signal of interest.

The principle can be understood using the Fourier shift theorem. Multiplication of a signal $f(t)$ with a complex exponential results in a shift of its spectrum

$$e^{i2\pi\xi_0 t} f(t) \iff \hat{f}(\xi - \xi_0)$$

which translates the spectrum by the frequency ξ_0.

In the present case, however, the signal of interest is a chirp signal whose frequency varies continuously over time. Therefore, the frequency shift must also be time-dependent. This is achieved by introducing a complex phasor

$$p(t) = e^{i\phi(t)}$$

where $\phi(t)$ represents the instantaneous phase of the chirp signal.

Multiplying the measured signal $x(t)$ with this phasor or its complex conjugate rotates the signal spectrum toward baseband

$$y_R(t) = x(t)p(t), \qquad y_Q(t) = x(t)p^*(t)$$

After this rotation step, the dominant signal component is centered around 0 Hz, which allows efficient filtering using a low-pass filter.

0.2.2.5. Baseband Filtering

Once the signal has been shifted to baseband, a low-pass filter can be applied to remove high-frequency noise components. Since the useful signal is concentrated near zero frequency, the filter can effectively suppress unwanted spectral components outside this region.

To avoid phase distortion, the filtering is performed using forward-backward filtering. This method applies the filter in both forward and reverse directions, resulting in zero-phase filtering.

The filtering operation produces the filtered baseband signals

$$y_R(t) \to \tilde{y}_R(t), \qquad y_Q(t) \to \tilde{y}_Q(t)$$

which contain the desired signal components with reduced noise.

0.2.2.6. Inverse Frequency Rotation

After filtering in the baseband, the signal is shifted back to its original frequency range. This is achieved by multiplying the filtered signals with the corresponding inverse phasors and combining the results to obtain a real-valued signal

$$x_f(t) = \mathrm{Re}\left(\frac{1}{2}\left(\tilde{y}_R(t)p^*(t) + \tilde{y}_Q(t)p(t)\right)\right).$$

This operation reverses the initial frequency rotation and reconstructs a filtered version of the original signal.

The resulting signal $x_f(t)$ retains the phase characteristics of the original signal while high-frequency noise components are effectively suppressed.

0.2.3. Control Loop Calculations

Although the flight controller consists of several nested control loops, their fundamental structure is identical. Each loop follows the same basic feedback control principle and differs mainly in the controlled variable and the chosen controller parameters. Therefore, instead of analysing every loop individually from the beginning, a simplified model is used to explain the underlying calculations and controller behaviour.

The simplified control structure used for the analysis consists of a controller C and a plant P, which describes the dynamic behaviour of the system being controlled.

Using this simplified model allows the general behaviour of the control loop and the calculation of the controller terms to be explained in a clear and structured way. The same principles are later applied to the individual control loops implemented in the flight controller.

0.2.3.1. Calculation of the Control Loop Components

After the signal processing steps, the control loop components can be calculated. As in the signal processing stage, the goal is to mitigate the influence of disturbances and noise on the calculated control loop components. Therefore, only the input signal of the control loop is used for the estimation.

The advantage of this approach is that

- the input signal is not affected by the back-calculation of the control loop and is therefore not influenced by disturbances or noise within the loop.
- the input signal is not affected by filtering steps, since it is not measured but taken directly from the software. Consequently, it is not subject to measurement noise or disturbances.

This approach allows a more precise estimation of the transfer function. However, it also means that the transfer functions of the individual control loop components cannot be calculated directly. Instead, they must be obtained through mathematical manipulations.

For example, consider a simplified control loop consisting of a controller $C(s)$ and a plant $P(s)$.

To calculate the transfer function of the controller $C(s)$, the following relation can be used

$$C(s) = \frac{Y(s)}{E(s)} = \frac{Y(s)}{R(s) - Y(s)} = \frac{Y(s)}{R(s)} \cdot \frac{1}{1 - \frac{Y(s)}{R(s)}} = \frac{G(s)}{1 - G(s)}.$$

To calculate the transfer function of the plant $P(s)$, the following relation can be used

$$P(s) = \frac{Y(s)}{U(s)} = \frac{Y(s)}{E(s)} \cdot \frac{E(s)}{U(s)} = C(s) \cdot \frac{1}{C(s)} = 1.$$

0.2.3.2. Closed-Loop Performance Metrics

After determining the plant transfer function, the next step is to analyze the behaviour of the overall system when a controller is applied. This is achieved by evaluating the closed-loop transfer functions, which describe the system response, disturbance sensitivity, disturbance rejection capability, and the required controller effort. These measures provide valuable insights into the stability, robustness, and performance of the controlled system.

The closed-loop transfer function from the reference input to the output is given by

$$G_{yr}(\omega) = \frac{C(\omega)P(\omega)}{1 + C(\omega)P(\omega)}.$$

The sensitivity function, which describes how sensitive the system output is to disturbances and model uncertainties, is defined as

$$S(\omega) = \frac{1}{1 + C(\omega)P(\omega)}.$$

The compliance function, representing the system response to disturbances acting on the plant, is given by

$$SP(\omega) = P(\omega)\, S(\omega).$$

Finally, the controller effort describes the influence of the reference input on the controller output and is defined as

$$SC(\omega) = C(\omega)\, S(\omega).$$

0.2.3.3. Step Response Analysis

To be added.

0.3. Method

In this chapter, the practical application of the theoretical concepts introduced in the previous chapter is described. The focus is on how these principles are implemented in the context of tuning the control loops of a Betaflight-based flight controller. The methodology is structured around the different control loops, starting with the angle controller, followed by the altitude hold controller and the position controller. For each control loop, the specific implementation details and tuning procedures are described.

0.3.1. Angle Mode

0.3.1.1. Difference between Angle and Acro Mode

The Betaflight software provides three main flight modes: Angle, Horizon, and Acro.

In Angle mode, the drone's tilt angle is directly linked to the stick position. The maximum tilt is limited to a predefined value, preventing flips and making this mode particularly suitable for beginners.

In contrast, Acro mode provides full manual control. In this mode, stick inputs define an angular rate rather than a target angle, meaning the drone continues rotating as long as the stick remains deflected. This allows aggressive maneuvers such as flips and rolls but also requires greater pilot experience.

During the project thesis, an offline tuner for the angular rate controller was further developed to systematically optimize the rate control loop and improve Acro mode performance. The tuner was subsequently extended to also support Angle mode, enabling improved stability and responsiveness while maintaining intuitive handling for beginner pilots.

0.3.1.2. Control Loops in Angle Mode

In Angle mode, the angular rate controller is extended by an outer angle control loop, resulting in a cascaded control structure.

The stick input initially generates a rate setpoint. Unlike in Acro mode, this rate setpoint is then converted into a target angle. This target angle serves as the reference input for the inner angular rate controller.

The cascaded architecture separates attitude control from angular rate stabilization. While the outer loop ensures convergence toward the desired orientation, the inner loop provides fast dynamic stabilization by regulating the angular velocity. As a result, the system achieves

improved disturbance rejection and more intuitive handling characteristics, since the pilot directly commands an angle rather than a rotational rate.

The Angle mode controller therefore consists of two cascaded loops:

- The outer angle loop, which computes the angle error
- The inner angular rate loop, which computes the rate error

The angle control loop consists of a proportional controller with an additional feedforward term. Within the offline tuning framework, however, only the proportional controller is optimized. The feedforward path is included in the conceptual description for completeness but was deliberately disabled during the tuning process.

0.3.1.3. Derivation of the Target Angle

The target angle is obtained by scaling the commanded angular rate with respect to the maximum achievable rate and the configured maximum tilt angle

$$\theta_{\text{target}} = \theta_{\text{limit}} \cdot \frac{\omega_{\text{set}}}{\omega_{\text{max}}} \tag{0.1}$$

where

- θ_{limit} is the maximum allowed tilt angle (e.g., $60°$),
- ω_{set} is the commanded angular rate derived from the stick input,
- ω_{max} is the maximum achievable angular rate determined by the rate configuration.

This relation ensures a proportional mapping between stick deflection and target angle.

For example,

$$\omega_{\text{set}} = \omega_{\text{max}} \Rightarrow \theta_{\text{target}} = \theta_{\text{limit}} \tag{0.2}$$

$$\omega_{\text{set}} = 0 \Rightarrow \theta_{\text{target}} = 0 \tag{0.3}$$

Thus, the stick position scales the target angle continuously within the range

$$-\theta_{\text{limit}} \leq \theta_{\text{target}} \leq +\theta_{\text{limit}} \tag{0.4}$$

List of Figures

List of Tables

1. Introduction

1.1. Background and Motivation

1.2. Research Questions and Objectives

1.3. Document Structure

This thesis is organized as follows. Chapter 2 provides the theoretical foundations... Chapter 3 describes the methodology... Chapter 4 details the implementation... Chapter 5 presents the experimental results... Chapter 6 discusses the findings and outlines future work.

2. Theoretical Foundations

2.1. Betaflight Control Architecture

2.2. System Identification via Chirp Excitation

2.3. Frequency Response Estimation

2.4. Closed-Loop Analysis

2.5. Cascaded Control Structures

3. Methodology

3.1. Tuning Workflow Overview

3.2. Flight Test Protocol

Hello, this is a placeholder for the flight test protocol section. Here we will describe the specific maneuvers and conditions under which the flight tests will be conducted to collect data for system identification and controller tuning.

3.3. Signal Processing Pipeline

3.4. Controller Design Procedure

3.5. Altitude Hold System Identification

4. Implementation

4.1. Software Architecture

4.2. MATLAB Implementation

4.3. Python Implementation

4.4. Betaflight Firmware Modifications

4.5. Validation Framework

5. Results

5.1. Rate Controller Tuning Results

5.2. Filter Configuration Results

5.3. Altitude Hold Identification Results

5.4. Python–MATLAB Consistency Validation

5.5. Flight Test Validation

6. Discussion and Outlook

6.1. Discussion of Results

6.2. Limitations

6.3. Future Work

6.4. Conclusion

Declaration of Generative AI Use

The following generative AI systems were used during the preparation of this thesis. Their use was limited to the functions described below. All generated content was critically reviewed and verified by the authors.

A. Project Management

A.1. Project Assignment

A.2. Project Schedule

B. Additional Figures and Data